题目 0：
Lecture 06 · Object-Oriented Programming 面向对象编程

> Advanced Programming — 张子邦（暨南大学光电工程系）
> OOP 概念、面向过程对比、类与对象、self、特殊方法（魔术方法）、仙剑奇侠传实战案例

一、什么是面向对象（OOP）

****Object-Oriented Programming（面向对象程序设计）**：一种围绕****对象**组织代码的编程方式——对象是自包含的"数据（属性 attributes）+ 行为（方法 methods）"共同体，而不是围绕独立的函数和全局数据。核心思想：****用程序模拟它所处理的现实世界事物**。

****"面向"（oriented）** = 朝向、以……为方向：
- 面向过程：以"****步骤/过程**"为中心组织程序
- 面向对象：以"****对象**"为基本单位思考和构建系统

****对象 = 状态（数据、变量）+ 行为（方法、函数）**，两者不可分割地绑定在一起：

对象	状态	行为
手机	电量、型号、使用时长	打电话、发消息、拍照
银行账户	余额	存款、取款、查余额
猫	名字、年龄、饥饿程度	叫、吃、睡

二、面向过程 vs 面向对象

	面向过程	面向对象
基本单位	函数/步骤	对象
数据与行为	分离	绑定在一起
思考方式	先做什么，再做什么	有哪些事物，它们如何协作
例子	`deposit(account, 100)`	`account.deposit(100)`

> 关键区别：面向过程是"****动作作用于数据**"，面向对象是"****对象自己完成动作**"。

****OOP 世界观**（理解面向对象最深刻的地方）：世界由具有自主行为的事物组成，复杂系统通过事物协作运转。以下单为例：
- 面向过程：一个大函数依次调用 检查库存 → 扣减库存 → 生成订单 → 扣款 → 发通知
- 面向对象：用户、购物车、商品、库存、订单、支付服务、通知服务各管各的事——购物车"结算"、库存"扣减"、订单"生成"、支付"扣款"、通知"发消息"，****下单就是对象互相发消息、协作的结果****

面向对象不是在"写流程"，而是在"****搭建一个由对象组成的小社会**"——每个对象有自己

的职责，通过接口（公有函数）沟通。

三、为什么需要 OOP

1960–70 年代软件规模快速增长，面向过程在大规模下崩溃：

问题 A：数据与行为分散——数据全局，任何函数都能修改：

```python
balance = 1000.0
def deposit(amt: float) -> None:
    global balance
    balance += amt
def withdraw(amt: float) -> None:
    global balance
    balance -= amt
def transfer(amt: float, target_account) -> None:
    global balance
    balance -= amt
```

任何函数都能改 `balance`，没人知道谁改了什么——**Bug 无法追踪**。

问题 B：代码重复——50 个"账户类"事物复制粘贴 50 遍，改一条规则要改 50 处。

问题 C：难以建模复杂系统——银行、仿真、GUI 中的事物天然有状态和行为，面向过程把每个"事物"拆成散落的变量和函数，丢失自然结构。

OOP 的回答：状态和行为打包进对象，隐藏内部细节，通过接口协作：

问题	OOP 解决方案
数据/行为分散	**封装（Encapsulation）**——数据存在对象内部
代码重复	**继承与组合（Inheritance & Composition）**——复用已有代码
复杂度建模难	**对象映射现实实体**

四、OOP 的六大实际好处

1. **可维护性（Maintainability）**：修改局部化——改 `Account.withdraw()` 规则只需改一处
2. **可复用性（Reusability）**：设计良好的类（Logger、Connection、Button）写一次到处用
3. **可扩展性（Scalability）**：大系统由小而独立的对象组成，团队可并行开发
4. **可拓展性（Extensibility）**：多态让你**不改现有代码**就增加新行为（新增

`SavingsAccount` 不用动 `Account`）
5. **安全性（Safety）**：封装阻止外部代码把对象置于非法状态（setter 禁止时无法 `balance = -999`）
6. **建模能力（Modeling Power）**：用领域概念（账户、订单、用户）写代码，**代码读起来像问题本身**

五、类与对象（Class vs Object）

进行 OOP 的主要工具：**类（class）**。类是创建对象的**蓝图（blueprint）**，定义：
- **属性（Attributes）**——对象持有的数据（状态）
- **方法（Methods）**——对象能执行的操作（行为）

类 ： 对象 = 数据类型（type）：变量（variable）

```python
guangzhou_temperature = Temperature()    # 一个类可以实例化出任意多个对象
tokyo_temperature     = Temperature()
cpu_temperature       = Temperature()
```

六、self 是什么

`self` 是**当前实例的引用**——对象通过它访问自己的属性和方法：

```python
class Dog:
    def __init__(self, name):
        self.name = name                # 存储到当前这个实例上
    def bark(self):
        print("%s says woof!" % (self.name))

d = Dog("Rex")
d.bark()                                # 输出：Rex says woof!
```

七、特殊方法（魔术方法 / dunder 方法）

双下划线包围的预定义方法（`__method__`），让自定义类与 Python 内置特性集成。

① 对象生命周期（Object Lifecycle）

方法	调用时机
`__new__(cls, ...)`	`__init__` 之前，**创建**实例

| `__init__(self, ...)` | 创建后，**初始化**实例 |
| `__del__(self)` | 对象即将被**销毁**时 |

```python
class Example:
    def __new__(cls, *args, **kwargs):
        print("Creating instance")
        return super().__new__(cls)
    def __init__(self, value):
        print("Initializing instance")
        self.value = value
    def __del__(self):
        print("Destroying instance")
```

② 字符串表示（String Representation）

方法	用途	触发方式
`__str__`	人类可读字符串	`print()`、`str()`
`__repr__`	无歧义字符串（调试用）	`repr()`、交互式环境

```python
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self):
        return "Point(%0.2f, %0.2f)" % (self.x, self.y)
    def __repr__(self):
        return "Point(x=%0.2f, y=%0.2f)" % (self.x, self.y)

p = Point(1, 2)
print(p)             # Point(1.00, 2.00)    ← 注意 %0.2f 保留两位小数
print(repr(p))   # Point(x=1.00, y=2.00)
```

③ 算术运算符重载（Arithmetic Operators）——让对象支持 `+` `-` `*` `/` 等运算：

方法	对应运算	方法	对应运算
`__add__`	`v1 + v2`	`__truediv__`	`v / 3`
`__sub__`	`v1 - v2`	`__neg__`	`-v`
`__mul__`	`v * 3`	`__abs__`	`abs(v)`
`__rmul__`	`3 * v`（反向乘法）		

```python
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):          return Vector(self.x + other.x, self.y + other.y)    # v1 + v2
    def __sub__(self, other):          return Vector(self.x - other.x, self.y - other.y)    # v1 - v2
    def __mul__(self, scalar):         return Vector(self.x * scalar, self.y * scalar)      # v * 3
    def __rmul__(self, scalar):        return self * scalar                                 # 3 *
v
    def __truediv__(self, scalar): return Vector(self.x / scalar, self.y / scalar)      # v / 3
    def __neg__(self):                 return Vector(-self.x, -self.y)                      # -v
    def __abs__(self):                 return (self.x**2 + self.y**2) ** 0.5                # abs(v)
```

> `__rmul__` 用于处理 `3 * v` 这种"数字在左、对象在右"的情况——Python 先尝试 `int.__mul__(3, v)` 失败后，再调用 `v.__rmul__(3)`。

八、课堂建模示例

三个类的骨架（属性 + 方法签名），用于课堂练习：

1. BankAccount（银行账户）

```python
class BankAccount:
    def __init__(self):
        self.id = None                 # 账号
        self._balance = None           # 余额
    def deposit(self, amount):       # 存款
        pass
    def withdraw(self, amount):      # 取款
        pass
    def getBalance(self):            # 查余额
        pass
```

2. Circle（圆）——周长 `C = 2πr`，面积 `S = πr²`；**半径应为正数**

```python
class Circle():
    def __init__(self):
        self.radius = None             # 半径
        self.x = None                  # 圆心坐标
```

```
        self.y = None
    def setRadius(self, radius):     # 设置半径（需校验为正）
        pass
    def getArea(self):                # 面积 πr²
        pass
    def getPerimeter(self):           # 周长 2πr
        pass
```

3. Temperature（温度）——三种温标换算：
- 摄氏 → 开尔文：`K = °C + 273.15`
- 摄氏 → 华氏：`°F = °C × 1.8 + 32`
- 华氏 → 开尔文：`K = (°F − 32) ÷ 1.8 + 273.15`
- **开尔文不能为负**（绝对零度限制）

```python
class Temperature:
    def __init__(self):
        self.value = None
    def setCelsius(self):             # 用摄氏度设置
        pass
    def getCelsius(self):             # 以摄氏度读取
        pass
    def setFahrenheit(self):          # 用华氏度设置
        pass
    def getFahrenheit(self):          # 以华氏度读取
        pass
    def setKelvin(self):              # 用开尔文设置（不能为负）
        pass
    def getKelvin(self):              # 以开尔文读取
        pass
```

九、综合案例：仙剑奇侠传（The Legend of Sword and Fairy）

用 OOP 对回合制 RPG 进行建模，将游戏实体抽象为三个类：**Role（角色）、Skill（法术技能）、Item（物品）**。

1. Role 人物角色类

按"身份 / 能力 / 资源 / 状态"四组组织属性，方法按职责分组：

```
Role
```

```
├── 属性
│   ├── 身份：name, level
│   ├── 能力：maxHp, maxMp, attack, defense, speed, luck, spirit
│   ├── 资源：hp, mp
│   └── 状态：skills, isAlive, isDefending
│
└── 方法
    ├── 派生：critRate, dodgeRate              # 由属性计算得出（暴击率、闪避率）
    ├── 打印：__str__, printStatus             # 状态展示
    ├── 资源：takeDamage, heal, restoreMp   # 承伤、回血、回蓝
    ├── 战斗：defend, attackTarget, useSkill
    ├── 结算：endTurn                          # 回合结束处理
    └── 成长：levelUp                          # 升级
```

属性	含义
`name`, `level`	姓名、等级（身份）
`maxHp`, `maxMp`	生命/法力上限
`attack`, `defense`	攻击、防御
`speed`, `luck`, `spirit`	速度、幸运、灵力
`hp`, `mp`	当前生命、法力（资源）
`skills`	掌握的技能列表
`isAlive`, `isDefending`	是否存活、是否处于防御状态

方法分组	方法	职责
派生属性	`critRate`、`dodgeRate`	由 luck/speed 等计算暴击率、闪避率
信息展示	`__str__`、`printStatus`	打印角色状态
资源变化	`takeDamage`、`heal`、`restoreMp`	受伤、治疗、恢复法力
战斗行动	`defend`、`attackTarget`、`useSkill`	防御、普攻目标、施放技能
回合结算	`endTurn`	每回合结束的状态更新
角色成长	`levelUp`	升级提升属性

2. Skill 法术技能类

```
Skill
├── 属性
│   ├── name          技能名
│   ├── mpCost        消耗法力
│   ├── power         威力系数
│   ├── skillType     类型（damage / heal / defend）
│   └── description  描述
```

```
│
└── 方法
    └── __init__      构造
```

`skillType` 用字符串区分技能用途：`damage`（伤害型）、`heal`（治疗型）、`defend`（辅助/防御型）。

3. Item 物品类

> 注：课件第 35 页 Item 类内容尚未完成（页面展示的仍是 Skill 的结构树），按课堂进度应为独立的物品类，用于封装药品/道具的名称、效果与使用逻辑，待课件更新后补充。

建模思路小结：Role 持有 `skills`（Skill 对象列表）体现了**对象之间可以组合协作**——角色施放技能（`useSkill`）时扣减自身 `mp`、调用 Skill 的 `power` 和 `skillType` 计算效果，这正是前面"对象组成的小社会"思想在游戏中的具体应用。

```
题目 1：Timer 类
"""Timer 计时器类
================================================================
设计理念
----------------------------------------------------------------
目的：对一段代码的运行时长进行计时。
用法：在代码段之前调用 start()，之后调用 end()，
      Timer 自动记录两个时刻并计算出中间经过的时长。

属性（成员变量）：
    _start    计时开始的时刻（秒），未开始时为 None
    _end      计时结束的时刻（秒），未结束时为 None
    running   计时器当前是否正在计时（防止重复 start/end）

方法（成员函数）：
    start()       记录开始时刻；若正在计时则报错
    end()         记录结束时刻；若未开始/已结束则报错
    elapsed()     返回计时时长（秒），内部按需换算
    __repr__()    打印计时结果，如 elapsed: 0.512 s

计时基准使用 time.perf_counter()——Python 中精度最高的
单调时钟，不受系统时间调整影响，专用于测量时间间隔。
================================================================
"""

import time
```

```
class Timer:
    def __init__(self):
        self._start = None     # 开始时刻（秒）
        self._end = None       # 结束时刻（秒）
        self.running = False   # 是否正在计时

    def start(self):
        """开始计时（在代码段之前调用）"""
        if self.running:
            raise Exception('timer is already running')
        self.running = True
        self._start = time.perf_counter()

    def end(self):
        """结束计时（在代码段之后调用）"""
        if not self.running:
            raise Exception('timer is not running')
        self._end = time.perf_counter()
        self.running = False

    def elapsed(self):
        """返回计时时长（秒）"""
        if self._start is None:
            raise Exception('timer has not been started')
        # 已结束则返回起止差值；仍在计时则返回当前经过的时间
        end = self._end if self._end is not None else time.perf_counter()
        return end - self._start

    def __repr__(self):
        return 'elapsed: %.3f s' % (self.elapsed())

# ---------------- 演示：测量一段代码的运行时长 ----------------
t = Timer()
t.start()                              # 代码段开始

total = 0                              # 被计时的代码段
for i in range(10 ** 7):
    total += i
print('total =', total)

t.end()                                # 代码段结束
print(t)                               # 输出计时时长
```

运行结果如下：

```
PS E:\Advanced programming> & E:/miniconda/python.exe "e:/Advanced prog
ramming/Timer.py"
total = 49999995000000
elapsed: 0.741 s
PS E:\Advanced programming>
```

题目 2：File 类

"""File 文件类

==========

设计理念

目的：对电脑中的"单个文件"建模。一个文件天然具有：

状态（路径、大小、修改时间……）+ 行为（读、写、删……），

适合用 OOP 封装——创建 File 对象即拿到该文件的"句柄"，

之后对文件的所有操作都通过对象方法完成，外部代码

不需要直接操作 os/open 等底层细节。

属性（成员变量）：

path	文件完整路径（字符串）
name	文件名（由 path 派生）
dir	所在目录（由 path 派生）
ext	扩展名（由 path 派生，如 '.txt'）

方法（成员函数）：

exists()	文件是否存在
getSize()	文件大小（字节）
getTime()	最后修改时间（'年-月-日 时:分:秒'）
read()	读取全部文本内容
write(text)	写入文本（覆盖）
append(text)	追加文本
delete()	删除文件
rename(new)	重命名（同步更新 path/name/ext 等属性）
__repr__()	打印文件摘要：name, size, time

封装要点：rename/delete 等方法在改变文件状态后同步更新

内部属性，保证对象状态始终与磁盘上的真实文件一致——

这正是"数据与行为绑定"的 OOP 思想。

==========

"""

```
import os
import time

class File:
```

```
def __init__(self, path):
    self.path = os.path.abspath(path)   # 统一存绝对路径
    self._updateInfo()

def _updateInfo(self):
    """由 path 派生 name / dir / ext（内部方法，属性变化后调用）"""
    self.dir = os.path.dirname(self.path)
    self.name = os.path.basename(self.path)
    self.ext = os.path.splitext(self.path)[1]

def exists(self):
    return os.path.isfile(self.path)

def getSize(self):
    """文件大小（字节）"""
    if not self.exists():
        raise Exception('file not found: %s' % (self.path))
    return os.path.getsize(self.path)

def getTime(self):
    """最后修改时间，格式：'2026-10-20 15:30:00'"""
    if not self.exists():
        raise Exception('file not found: %s' % (self.path))
    stamp = os.path.getmtime(self.path)
    return time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(stamp))

def read(self):
    """读取全部文本"""
    with open(self.path, 'r', encoding='utf-8') as f:
        return f.read()

def write(self, text):
    """写入文本（覆盖原内容），创建后不存在的文件"""
    with open(self.path, 'w', encoding='utf-8') as f:
        f.write(text)

def append(self, text):
    """在末尾追加文本"""
    with open(self.path, 'a', encoding='utf-8') as f:
        f.write(text)

def delete(self):
    """删除文件"""
    if not self.exists():
```

```
            raise Exception('file not found: %s' % (self.path))
        os.remove(self.path)

    def rename(self, new_name):
        """重命名文件，并同步更新内部属性"""
        if not self.exists():
            raise Exception('file not found: %s' % (self.path))
        new_path = os.path.join(self.dir, new_name)
        os.rename(self.path, new_path)
        self.path = new_path
        self._updateInfo()

    def __repr__(self):
        if not self.exists():
            return 'File(%s) [not found]' % (self.name)
        return 'File(%s, %d bytes, %s)' % (self.name, self.getSize(), self.getTime())

# ---------------- 演示：对作业文件夹中的文件建模 ----------------
f = File(r'作业\hello_world.py')
print(f)                                # 文件摘要
print('name:', f.name)
print('dir :', f.dir)
print('ext :', f.ext)
print('size:', f.getSize(), 'bytes')
print('time:', f.getTime())
print('content:')
print(f.read())

# 写入 / 追加 / 重命名 / 删除 演示（用临时文件）
tmp = File('_file_test.txt')
tmp.write('Hello, File class!\n')
tmp.append('written by append()\n')
print('tmp size:', tmp.getSize(), 'bytes')
print(tmp.read())
tmp.rename('_file_test_renamed.txt')
print('renamed to:', tmp.name)
tmp.delete()
print('exists after delete:', tmp.exists())
```

运行结果如下：

```
PS E:\Advanced programming> & E:/miniconda/python.exe "e:/Advanced prog
ramming/File.py"
File(hello_world.py, 22 bytes, 2026-09-15 21:15:22)
name: hello_world.py
dir : E:\Advanced programming\作业
ext : .py
size: 22 bytes
time: 2026-09-15 21:15:22
content:
print('Hello, World!')
tmp size: 41 bytes
Hello, File class!
written by append()

renamed to: _file_test_renamed.txt
exists after delete: False
PS E:\Advanced programming>
```